Inteligencia Artificial

Estado del Arte: Problema Generación Procedural de Plataformas

Bruno Morici

9 de octubre de 2026

Evaluación

Resumen (5 %):	_____
Introducción (5 %):	_____
Definición del Problema (10 %):	_____
Estado del Arte (40 %):	_____
Modelo Matemático (20 %):	_____
Conclusiones (15 %):	_____
Bibliografía (5 %):	_____
Nota Final (100 %):	_____

Resumen

En el presente documento se estudia la generación procedural para videojuegos de plataformas en dos dimensiones, un caso particular de la generación procedural de contenido (PCG). El problema consiste en determinar automáticamente la disposición de los elementos de un nivel, de forma que el resultado pueda ser completado según las capacidades físicas del ávatar y, a la vez, se ajuste a los parámetros definidos por el diseñador del nivel (dificultad, estilo, ritmo). Luego se revisa el estado del arte, considerando diferentes enfoques, y se comparan sus representaciones, resultados y limitaciones. Finalmente se propone un modelo matemático que formula el problema como un problema de satisfacción de restricciones con optimización (CSOP), definiendo sus variables, restricciones y función objetivo.

1. Introducción

La generación procedural de contenido (PCG) corresponde a la creación automática de contenido para videojuegos mediante algoritmos [4]. En los videojuegos de plataformas en dos dimensiones, como Super Mario Bros, Donkey Kong o Geometry Dash, generar un nivel implica decidir la disposición del suelo, plataformas, enemigos, huecos, obstáculos y recompensas de modo que el jugador pueda recorrerlo desde el inicio hasta la meta con caminos factibles.

La industria de los videojuegos superó en 2022 los ingresos de la música y el cine, siempre necesita contenido nuevo [4], sin embargo, la generación de contenido representa un costo significativo en la producción de los videojuegos, impactando su presupuesto y tiempo de desarrollo.

La PCG permite reducir este esfuerzo y aumentar la rejugabilidad, minimizando el esfuerzo para los desarrolladores y diseñadores de niveles.

El propósito de este documento es analizar el estado del arte de la PCG en niveles de plataformas 2D, comparando enfoques constructivos, basados en búsqueda, basados en restricciones y de aprendizaje automático. Como contribución, se presenta una formulación matemática del problema como un problema de satisfacción de restricciones con optimización (CSOP), basada en el modelo de restricciones propuesto por Cooper [1], integrando la jugabilidad como restricción y los criterios del diseñador como función objetivo.

El documento se organiza de la siguiente forma. La Sección 2 define el problema, sus variables, restricciones, objetivos y variantes. La Sección 3 presenta el estado del arte, describiendo los principales enfoques, las representaciones utilizadas y algoritmos utilizados. La Sección 4 propone el modelo matemático. La Sección 5 expone las conclusiones, y finalmente se presenta la bibliografía junto con el detalle del uso de LLMs.

2. Definición del Problema

Para describir el problema de generación procedural de contenido enfocado en niveles de videojuegos de plataformas en dos dimensiones, se utiliza como referencia el caso de *Super Mario Bros.* (Nintendo, 1985). El problema que surge en este tipo de videojuegos es cómo colocar las diferentes plataformas, obstáculos, enemigos, monedas y resortes dentro de un nivel, de modo que el resultado se acerque lo más posible a los parámetros entregados por el diseñador del nivel y que, a su vez, el nivel sea jugable, es decir, que pueda completarse desde el inicio hasta la meta. En estos videojuegos se definen ciertos elementos relevantes para el problema [7]:

- **Avatar:** personaje controlado por el jugador. Sus capacidades de movimiento (caminar, correr y saltar) determinan qué partes del nivel son alcanzables.
- **Plataformas:** superficies sobre las cuales el avatar puede desplazarse de forma segura.
- **Obstáculos:** cualquier elemento que dañe al avatar o interrumpa el flujo del juego. Los **enemigos**, que el jugador debe eliminar o evitar, y los **huecos**, espacios sin suelo cuya caída implica perder una vida, son tipos de obstáculo.
- **Ayudas de movimiento:** elementos como los resortes, que modifican temporalmente la velocidad o dirección del avatar.
- **Recompensas:** elementos como las monedas, que premian al jugador y a la vez lo guían a lo largo del nivel.

Un nivel jugable constituye una solución de este problema. Muchas de las soluciones algorítmicas existentes dividen el problema en dos partes: la creación de escenas y la agrupación de escenas [7, 2, 5]. La creación de escenas consiste en construir segmentos cortos de un nivel, cada uno con un inicio y un fin, que deben poder anexarse entre sí; estas escenas pueden ser creadas a mano por los diseñadores [5] o mediante un algoritmo, por ejemplo, uno basado en el ritmo de acciones del jugador [7]. La agrupación de escenas, por su parte, es el procedimiento mediante el cual se anexan las escenas previamente creadas para formar un nivel completo. No obstante, también existen enfoques que generan el nivel completo casilla por casilla [1] o que ajustan los parámetros globales de un generador [6].

Independientemente del enfoque, surgen dos preguntas: qué se entiende por un nivel de calidad y cómo garantizan los algoritmos que el nivel resultante la tenga. Para responderlas, a continuación se detallan las dificultades, variables, restricciones, objetivo y variantes del problema.

2.1. Dificultades

La principal dificultad para la resolución de este problema surge de que la cantidad de niveles posibles crece exponencialmente con el tamaño del nivel y con el número de tipos de elementos, por lo que no es factible revisarlos todos. A esto se suman otras dificultades:

- **La jugabilidad es una condición global:** no basta con que cada elemento sea compatible con sus vecinos; es necesario verificar que exista un camino completo desde el inicio hasta la meta [3], lo que suele requerir simular el movimiento del avatar o hacer que un agente juegue el nivel [2].
- **La calidad es difícil de medir:** conceptos como la diversión o el desafío son subjetivos, y los modelos que intentan predecirlos tienen una precisión limitada [6].
- **Equilibrio entre variedad y control:** otorgar mayor libertad al generador produce niveles más variados, pero aumenta el riesgo de obtener niveles injugables [5].

2.2. Variables

Las variables del problema corresponden a las decisiones que debe tomar el generador:

- Qué elemento (suelo, aire, enemigo, moneda, resorte, entre otros) ocupa cada casilla del nivel, en los enfoques que lo representan como una grilla [1].
- Qué escenas se seleccionan y en qué orden se anexan, en los enfoques basados en escenas [2, 5].

Estas decisiones dependen de parámetros que no cambian durante la generación:

- Las dimensiones del nivel (alto y ancho).
- Las capacidades físicas del avatar, como la altura y la distancia máxima de salto [7].
- Las preferencias del diseñador, como la cantidad o frecuencia deseada de cada elemento, la dificultad o la trayectoria general del nivel [7].

2.3. Restricciones

Para que un nivel sea válido, debe cumplir las siguientes restricciones [1]:

- Cada casilla contiene exactamente un elemento.
- El nivel tiene exactamente un inicio y una meta.
- Los enemigos y obstáculos se ubican sobre una plataforma o sobre el suelo.
- Los elementos vecinos forman combinaciones válidas, similares a las de niveles de ejemplo; por ejemplo, una tubería no puede quedar incompleta.
- La separación y la diferencia de altura entre plataformas respetan las capacidades de salto del avatar [7].
- **Jugabilidad:** debe existir un camino desde el inicio hasta la meta que el avatar pueda recorrer caminando, cayendo o saltando.

Los algoritmos difieren en cómo tratan estas restricciones: algunos las garantizan por construcción [7], otros las verifican posteriormente mediante agentes que juegan el nivel [2] y otros no garantizan la jugabilidad [5].

2.4. Objetivo

El objetivo es obtener un nivel jugable en el que la cantidad de cada tipo de elemento sea lo más cercana posible a la que pidió el diseñador, es decir, minimizar la diferencia entre la cantidad solicitada y la cantidad colocada de cada elemento [1, 7].

2.5. Variantes

Entre las variantes más conocidas del problema está la generación personalizada, donde el nivel se adapta al jugador mientras juega [6]; la generación a partir de un nivel de referencia, que busca repetir su secuencia de mecánicas con un nivel distinto [2]; la generación basada en ritmo [7]; el ensamblaje de piezas hechas a mano [5]; y la reparación o unión de niveles ya existentes [1]. También existen variantes con otros tipos de movimiento, como juegos sin salto estándar [1], y la generación de niveles en 3D, que todavía está poco estudiada [4].

3. Estado del Arte

3.1. Origen del problema

La PCG se usa en videojuegos desde fines de los años 70, sobre todo en juegos *roguelike* como *Beneath Apple Manor* (1978) y *Rogue* (1980), que generaban mazmorras distintas en cada partida [4]. En los juegos de plataformas, en cambio, los intentos de generar niveles completos fueron pocos hasta mediados de los 2000 [6]. Juegos como *Spelunky* usaron piezas prediseñadas ubicadas en una grilla, lo que funciona porque el jugador puede modificar el nivel con bombas o cuerdas, pero no sirve para niveles lineales como los de *Super Mario Bros.* [7]. Más adelante, con *Infinite Mario Bros*, un clon libre de *Super Mario Bros.*, y las competencias *Mario AI* desde 2009, este juego pasó a ser el entorno de prueba más usado para estudiar el problema [6, 2].

3.2. Launchpad: generación de escenas basada en ritmo

Smith et al. [7] proponen Launchpad, un generador basado en un modelo del diseño de niveles de plataformas. Su principal avance es que no une trozos de nivel hechos a mano, sino que construye las escenas a partir de los componentes básicos (plataformas, huecos, enemigos y resortes), por lo que casi no requiere trabajo manual.

Cada escena es un *grupo de ritmo*, un tramo corto del nivel con un desafío. Primero se genera el ritmo, que es la secuencia de acciones del jugador en el tiempo (moverse o saltar), con un tipo, una densidad y una duración de entre 5 y 20 segundos. Después, una gramática convierte cada acción en geometría; por ejemplo, un salto puede transformarse en un hueco, un enemigo o un resorte. La jugabilidad se asegura por construcción: un modelo físico del avatar indica cuánto tiempo pasa en el aire con cada tipo de salto, y si una geometría no le permite aterrizar antes de la siguiente acción del ritmo, esa opción se descarta.

Las escenas se unen con plataformas de descanso, que son planas y no tienen desafíos, por lo que el nivel completo sigue siendo jugable. Al final se agrega un suelo común y se reparten monedas por el nivel. Como el algoritmo puede generar muchos niveles distintos, Launchpad aplica la técnica de *generar y probar*: crea muchos candidatos y elige el mejor según qué tan cerca está de una trayectoria definida por el diseñador y de la frecuencia de componentes que este pidió. Como limitaciones, solo genera niveles de un único camino y su modelo físico no considera mecánicas como el doble salto.

3.3. Scene Stitching: unión de escenas mediante búsqueda evolutiva

Green et al. [2] no generan escenas, sino que resuelven cómo unirlas. Usan un conjunto de 1691 escenas de *Super Mario Bros.* creadas previamente, cada una marcada con las mecánicas

que el jugador debe usar para pasarla. A partir de la secuencia de mecánicas que un agente usa al jugar un nivel original, buscan una secuencia de escenas que entregue una experiencia parecida, pero con un nivel distinto.

La búsqueda se hace con FI-2Pop, un algoritmo genético con dos poblaciones: una de niveles infactibles, que evoluciona para cumplir la jugabilidad, y otra de niveles factibles, que evoluciona para acercarse a la secuencia de mecánicas buscada. Cada individuo es una lista de escenas de largo variable, que se modifica con cruce de dos puntos y con mutaciones que agregan, quitan, dividen, juntan o cambian escenas. Aquí la jugabilidad no se asegura por construcción, sino que se revisa después con un agente externo: un agente A* juega cada nivel varias veces y, si gana en suficientes partidas, el nivel se considera jugable; si no, se mide cuánto alcanzó a avanzar. Con este método, entre un 87,25 % y un 100 % de los niveles resultaron completables, contra un 25–28,5 % de un método voraz. Como limitaciones, los niveles quedan más planos y menos variados que los originales, y jugar cada nivel con el agente es costoso.

3.4. Otros enfoques

Mawhorter y Mateas [5] arman niveles uniendo piezas hechas a mano a partir de anclas, que son posiciones que el jugador puede ocupar; el método da niveles variados, pero no asegura que sean jugables. Shaker et al. [6] adaptan los niveles a cada jugador: con datos de 327 jugadores entrenan redes neuronales que predicen la diversión según cuatro parámetros del nivel y el estilo del jugador, y luego eligen los parámetros que dan más diversión. En sus pruebas, el 60 % de los participantes prefirió el nivel adaptado, aunque la jugabilidad no se modela.

Por último, Cooper [1] propone Sturgeon, que plantea la generación como un problema de restricciones sobre variables booleanas y lo resuelve con *solvers* como SAT o ASP. El nivel se arma casilla por casilla usando patrones aprendidos de un nivel de ejemplo y una restricción de alcanzabilidad que exige un camino desde el inicio hasta la meta. En *Super Mario Bros.* genera niveles jugables en segundos y también permite reparar o completar niveles. Este enfoque se parece a *WaveFunctionCollapse* (WFC), que según Karth y Smith [3] también resuelve restricciones, pero solo entre casillas vecinas, por lo que no asegura que exista un camino completo por el nivel.

3.5. Clasificación de los algoritmos

Maleki y Zhao [4] revisan 207 artículos publicados entre 2019 y 2023 y separan los algoritmos de PCG en cinco grupos: (1) métodos de búsqueda, que ven el problema como uno de optimización, como los algoritmos evolutivos usados en Scene Stitching; (2) aprendizaje automático, que aprende de niveles existentes para generar otros parecidos, como las redes neuronales o el aprendizaje por refuerzo; (3) otros métodos constructivos, como el ruido o las gramáticas, grupo en el que entra Launchpad; (4) modelos de lenguaje (LLM), como MarioGPT, que genera niveles de Mario a partir de texto; y (5) métodos combinados, que mezclan los anteriores, por ejemplo, para reparar niveles. En este documento se agrega un sexto grupo, el de los métodos basados en restricciones, como Sturgeon [1] y WFC [3], ya que son los únicos que tratan la jugabilidad como una restricción explícita.

3.6. Representaciones y resultados

La Tabla 1 resume los enfoques revisados según su representación del nivel y cómo tratan la jugabilidad.

Cada representación tiene ventajas distintas. El ritmo con geometría asegura la jugabilidad de forma directa, pero solo permite niveles de un camino [7]. Las escenas o piezas reducen la búsqueda, pero dependen de una biblioteca previa que limita la variedad [5, 2]. El vector de parámetros es simple y rápido, pero solo controla aspectos generales del nivel [6]. La grilla de

Trabajo	Enfoque	Representación	Jugabilidad	Objetivo
Smith et al. [7]	Gramática con generar y probar	Ritmo y geometría	Garantizada por construcción	Críticos del diseñador
Mawhorter y Mateas [5]	Ensamblaje de piezas	Piezas con anclas	No garantizada	Variedad
Shaker et al. [6]	Modelo del jugador y búsqueda exhaustiva	Vector de 4 parámetros	No modelada	Maximizar la diversión predicha
Green et al. [2]	Evolutivo (FI-2Pop)	Secuencia de escenas	Verificada con agente A*	Similitud de mecánicas
Cooper [1]	Satisfacción de restricciones	Grilla de tiles	Garantizada como restricción	Distribución de tiles

Cuadro 1: Comparación de enfoques para la generación de niveles de plataformas 2D.

tiles es la más flexible y permite escribir la jugabilidad como una restricción, aunque su espacio de búsqueda es mucho mayor [1]; por eso es la que se usa en el modelo de este documento. Como cada trabajo se evalúa con métricas y juegos distintos, no hay una forma común de decir qué algoritmo es mejor.

3.7. Tendencia actual

La generación de niveles es el tema más estudiado de la PCG, con 102 de los 207 artículos revisados, y *Super Mario Bros.* es el juego más usado [4]. Entre 2019 y 2023 los métodos más usados fueron los clásicos, como el ruido y las gramáticas, seguidos por el aprendizaje automático, mientras que los LLM pasaron de uno o dos artículos al año a 13 en 2023 [4]. La investigación apunta a combinar métodos, a reparar niveles en vez de generarlos desde cero y a la generación en 3D [4].

4. Modelo Matemático

El modelo presentado a continuación corresponde a una adaptación del sistema Sturgeon [1], cuyas restricciones se describen originalmente mediante una interfaz de programación y aquí se expresan en notación matemática, aplicadas al caso de *Super Mario Bros.* La función objetivo es una adaptación propia, inspirada en las reglas de distribución de Sturgeon y en el crítico de frecuencia de componentes de Launchpad [7]. El problema se formula como un **problema de satisfacción de restricciones con optimización (CSOP)**: las restricciones definen qué niveles son válidos y la función objetivo selecciona, entre ellos, el que mejor se ajusta a lo solicitado por el diseñador.

4.1. Conjuntos y parámetros

- $I = \{1, \ldots, H\}$: filas del nivel, donde la fila 1 es la superior. En *Super Mario Bros.*, $H = 14$.
- $J = \{1, \ldots, W\}$: columnas del nivel, de izquierda a derecha.
- T: tipos de tile. Para Mario se considera $T = \{\text{aire, sólido, tubería, enemigo, moneda, inicio, meta}\}$.
- $T_c = \{\text{sólido}, \text{tubería}\} \subset T$: tiles **cerrados**, que el avatar no puede atravesar y sobre los cuales puede pararse. El resto, $T_a = T \setminus T_c$, son tiles **abiertos**.
- Π: conjunto de patrones permitidos, formado por todas las secuencias de tres columnas consecutivas observadas en un nivel de ejemplo (por ejemplo, el nivel 1-1 de *Super Mario Bros.*), equivalente a la plantilla *3gr-col* de Cooper.

- $G = (V, E)$: grafo de movimiento. Cada nodo $u \in V$ es una casilla (i, j) y cada arista $e = (u, v) \in E$ representa un movimiento potencial del avatar (caminar, caer o saltar), definido según su altura máxima de salto h_s y su distancia máxima de salto d_s. Cada arista tiene asociados dos conjuntos de casillas: $A(e)$, que deben estar abiertas (por ejemplo, las del arco de un salto), y $B(e)$, que deben estar cerradas (por ejemplo, la casilla bajo el punto de partida).
- $\text{in}(v)$ y $\text{out}(v)$: aristas que llegan a la casilla v y que salen de ella.
- k: ancho, en columnas, de las zonas válidas para el inicio y la meta.
- n_t: cantidad de tiles de tipo t solicitada por el diseñador, para cada $t \in T_d \subseteq T$, donde T_d es el conjunto de tipos controlados (por ejemplo, enemigos, monedas y tuberías).
- $w_t \geq 0$: importancia relativa de cumplir la cantidad solicitada del tipo t.

4.2. Variables y dominios

- $x_{i,j,t} \in \{0, 1\}$: vale 1 si la casilla (i, j) contiene un tile de tipo t, y 0 en caso contrario. Es la variable principal del modelo. De forma equivalente, puede definirse una variable categórica $X_{i,j} \in T$ que indica el tipo de tile de cada casilla, donde $X_{i,j} = t \Leftrightarrow x_{i,j,t} = 1$.
- $a_u \in \{0, 1\}$: vale 1 si la casilla u está abierta, es decir, si contiene un tile de T_a.
- $r_u \in \{0, 1\}$: vale 1 si la casilla u forma parte del camino recorrido desde el inicio hasta la meta.
- $z_e \in \{0, 1\}$: vale 1 si el movimiento representado por la arista e se utiliza en ese camino.
- $\delta_t \in \mathbb{R}_{\geq 0}$: diferencia, en valor absoluto, entre la cantidad de tiles de tipo t colocada y la solicitada, para cada $t \in T_d$.

4.3. Espacio de búsqueda

Como cada casilla puede contener cualquiera de los $|T|$ tipos de tile, el número de niveles posibles es $|T|^{H \cdot W}$, antes de aplicar las restricciones. Las variables a, r, z y δ quedan determinadas, o acotadas, por la elección de x. Para un nivel de 14 filas y 60 columnas con los 7 tipos de tile definidos, el espacio contiene $7^{840} \approx 10^{710}$ niveles posibles, lo que impide una búsqueda exhaustiva y justifica el uso de propagación de restricciones y *solvers* especializados.

4.4. Función objetivo

$$\text{mín } Z = \sum_{t \in T_d} w_t \, \delta_t \tag{1}$$

donde δ_t se relaciona con la cantidad colocada mediante las siguientes restricciones, que linealizan el valor absoluto:

$$\delta_t \geq \sum_{i \in I} \sum_{j \in J} x_{i,j,t} - n_t \qquad \forall t \in T_d \tag{2}$$

$$\delta_t \geq n_t - \sum_{i \in I} \sum_{j \in J} x_{i,j,t} \qquad \forall t \in T_d \tag{3}$$

En palabras, se busca minimizar la suma ponderada de las diferencias entre la cantidad de cada tipo de elemento solicitada por el diseñador y la cantidad realmente colocada en el nivel. Las restricciones (2) y (3) obligan a que δ_t sea al menos la diferencia en ambos sentidos, por lo que, al minimizar, δ_t toma exactamente el valor absoluto de esa diferencia.

4.5. Restricciones

1. Un único tile por casilla.

$$\sum_{t \in T} x_{i,j,t} = 1 \qquad \forall i \in I,\ j \in J \tag{4}$$

Cada casilla del nivel contiene exactamente un tipo de tile. Corresponde a las reglas de tile de Cooper [1].

2. Un inicio y una meta en posiciones válidas.

$$\sum_{i \in I} \sum_{j \in J} x_{i,j,\text{inicio}} = 1, \qquad \sum_{i \in I} \sum_{j \in J} x_{i,j,\text{meta}} = 1 \tag{5}$$

$$x_{i,j,\text{inicio}} = 0 \quad \forall j > k, \qquad x_{i,j,\text{meta}} = 0 \quad \forall j \leq W - k \tag{6}$$

El nivel tiene exactamente un punto de inicio, ubicado en las primeras k columnas, y exactamente una meta, ubicada en las últimas k columnas. Corresponde a las restricciones de inicio y meta de las reglas de alcanzabilidad de Cooper; las zonas válidas se adaptan al recorrido de izquierda a derecha de Mario.

3. Patrones locales.

$$(X_{\cdot,j},\ X_{\cdot,j+1},\ X_{\cdot,j+2}) \in \Pi \qquad \forall j \in \{1, \ldots, W-2\} \tag{7}$$

donde $X_{\cdot,j}$ es la columna j completa del nivel. Esta restricción revisa el nivel de a tres columnas seguidas y exige que cada uno de esos grupos ya haya aparecido en un nivel de ejemplo de *Super Mario Bros.* Así, el nivel generado se ve como uno del juego original y se evitan cosas que no tienen sentido, como una tubería cortada por la mitad. Se basa en las reglas de patrón de Cooper [1].

4. Casillas abiertas.

$$a_u = \sum_{t \in T_a} x_{u,t} \qquad \forall u \in V \tag{8}$$

Una casilla se considera abierta si y solo si contiene un tile que el avatar puede atravesar.

5. Validez de los movimientos.

$$z_e \leq a_c \qquad \forall e \in E,\ c \in A(e) \tag{9}$$

$$z_e \leq 1 - a_c \qquad \forall e \in E,\ c \in B(e) \tag{10}$$

Un movimiento solo puede utilizarse si las casillas que exige abiertas están abiertas y las que exige cerradas están cerradas. Por ejemplo, un salto solo es posible si el avatar parte desde encima de un tile cerrado y todas las casillas del arco del salto están libres.

6. Continuidad del camino.

$$z_e \leq r_u \qquad \forall e = (u, v) \in E \tag{11}$$

$$\sum_{e \in \text{in}(v)} z_e \leq 1, \qquad \sum_{e \in \text{out}(v)} z_e \leq 1 \qquad \forall v \in V \tag{12}$$

$$r_v \leq x_{v,\text{inicio}} + \sum_{e \in \text{in}(v)} z_e \qquad \forall v \in V \tag{13}$$

Solo puede salir un movimiento desde una casilla que pertenece al camino; cada casilla tiene a lo más un movimiento de entrada y uno de salida; y una casilla que no es el inicio solo pertenece al camino si se llega a ella mediante algún movimiento.

7. El camino une el inicio con la meta.

$$x_{v,\text{inicio}} \leq r_v, \qquad x_{v,\text{inicio}} + z_e \leq 1 \quad \forall e \in \text{in}(v) \qquad \forall v \in V \tag{14}$$

$$x_{v,\text{meta}} \leq r_v, \qquad x_{v,\text{meta}} + z_e \leq 1 \quad \forall e \in \text{out}(v) \qquad \forall v \in V \tag{15}$$

La casilla de inicio pertenece al camino y no recibe movimientos, y la casilla de la meta pertenece al camino y no tiene movimientos de salida. Junto con la restricción 6, esto obliga a que exista una secuencia de movimientos válidos que parte en el inicio y termina en la meta: es la **restricción de jugabilidad**. Las restricciones 4 a 7 corresponden a las reglas de alcanzabilidad de Cooper con la plantilla de movimiento para juegos de plataformas.

8. Apoyo de enemigos (adición no presente en Cooper).

$$x_{i,j,\text{enemigo}} \leq \sum_{t \in T_c} x_{i+1,j,t} \quad \forall i \in \{1, \ldots, H-1\},\ j \in J, \qquad x_{H,j,\text{enemigo}} = 0 \quad \forall j \in J \tag{16}$$

Todo enemigo debe ubicarse directamente sobre un tile cerrado, como el suelo o una plataforma, y por lo tanto no puede estar en la última fila. En Sturgeon esta condición no se impone explícitamente, sino que emerge de los patrones aprendidos; aquí se agrega como restricción para asegurar este comportamiento propio de *Super Mario Bros.*

Resumen del modelo. El problema consiste en encontrar valores para x, a, r, z y δ que minimicen (1) sujeto a las restricciones 1 a 8. Las restricciones 4 a 7 en conjunto aseguran la jugabilidad del nivel: definen qué movimientos son posibles y exigen que exista una cadena de ellos desde el inicio hasta la meta.

5. Conclusiones

Las técnicas revisadas no resuelven exactamente el mismo problema planteado en este documento: Launchpad crea y une escenas [7], Scene Stitching solo las une [2], Shaker et al. ajustan parámetros [6] y Sturgeon genera el nivel casilla por casilla [1], siendo este último el más cercano al modelo propuesto. Todas se parecen en que buscan niveles jugables y en que usan alguna medida de calidad definida por el diseñador o el jugador. Lo que más las diferencia es cómo aseguran la jugabilidad, ya sea por construcción, revisándola con un agente o como una restricción, y cada una tiene sus limitaciones: Launchpad solo genera niveles de un camino, Scene Stitching depende de escenas ya creadas y de un agente costoso, Shaker et al. no modelan la jugabilidad y Sturgeon se vuelve lento en niveles grandes o al reparar niveles.

Una idea para seguir investigando es juntar ambas partes del problema: crear escenas jugables a partir del ritmo, como en Launchpad, y unirlas con un modelo de restricciones como el de este documento, para asegurar la jugabilidad del nivel completo sin tener que jugarlo con un agente.

6. Uso LLMs

1. Herramienta utilizada: Claude (Modelo Opus 5.5 en formato Alto consumo), Plan Claude Pro de Anthropic.

2. Propósito de uso: Citar las referencias en LaTex, encontrar ideas de referencias para indagar, resumir en PDF las referencias, apoyo en fórmulas LaTex para el modelo matemático.

3. Prompts relevantes: Resume en español estos papers de PCG. Expresa el modelo de Sturgeon en notación matemática. Busca papers complementarios.

4. Nivel de edición: Se copió y pegó las citas en la bibliografía con una revisión posterior para confirmar correctitud de formato APA y reglas de citado de cada referencia, al igual que el modelo matemático, el cual fue revisado y corregido posteriormente.

5. Validación: Uso de papers, apuntes de clase, juicio propio.

Referencias

[1] Seth Cooper. Sturgeon: Tile-based procedural level generation via learned and designed constraints. In *Proceedings of the Eighteenth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 26–36. AAAI Press, 2022. `https://doi.org/10.1609/aiide.v18i1.21944`.

[2] Michael Cerny Green, Luvneesh Mugrai, Ahmed Khalifa, and Julian Togelius. Mario level generation from mechanics using scene stitching. In *2020 IEEE Conference on Games (CoG)*, pages 49–56, Osaka, Japan, 2020. IEEE. Preprint: `https://arxiv.org/abs/2002.02992`.

[3] Isaac Karth and Adam M. Smith. WaveFunctionCollapse is constraint solving in the wild. In *Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG '17)*, pages 68:1–68:10, Hyannis, MA, USA, 2017. ACM. `https://doi.org/10.1145/3102071.3110566`.

[4] Mahdi Farrokhi Maleki and Richard Zhao. Procedural content generation in games: A survey with insights on emerging LLM integration. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 167–178. AAAI Press, 2024. `https://doi.org/10.1609/aiide.v20i1.31877`.

[5] Peter A. Mawhorter and Michael Mateas. Procedural level generation using occupancy-regulated extension. In Georgios N. Yannakakis and Julian Togelius, editors, *Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 351–358, Copenhagen, Denmark, 2010. IEEE.

[6] Noor Shaker, Georgios N. Yannakakis, and Julian Togelius. Towards automatic personalized content generation for platform games. In *Proceedings of the Sixth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 63–68, Palo Alto, CA, USA, 2010. AAAI Press. `https://doi.org/10.1609/aiide.v6i1.12399`.

[7] Gillian Smith, Jim Whitehead, Michael Mateas, Mike Treanor, Jameka March, and Mee Cha. Launchpad: A rhythm-based level generator for 2-D platformers. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(1):1–16, 2011. `https://doi.org/10.1109/TCIAIG.2010.2095855`.